

Advanced Encryption Schemes - Lecture 5

Cecilia Boschini and Kristina Hostáková

Outline

Recap: Lecture 4

Key idea

Technical tools

- Subset membership problems
- Smooth Projective Hashing

IND-CPA from a Smooth Projective Hash

Three security notions for PKE

Same game (guess b from $ct^* \leftarrow \mathrm{Enc}(pk, m_b)$); they differ only in the access to $\mathcal{O}_{\mathrm{Dec}}(\cdot) = \mathrm{Dec}(sk, \cdot)$:

	$\mathcal{O}_{\mathrm{Dec}}$ *before* ct^*	$\mathcal{O}_{\mathrm{Dec}}$ *after* ct^*	Adversary
IND-CPA	×	×	passive
IND-CCA1	✓	×	lunchtime
IND-CCA2	✓	✓ $ct \neq ct^*$	active

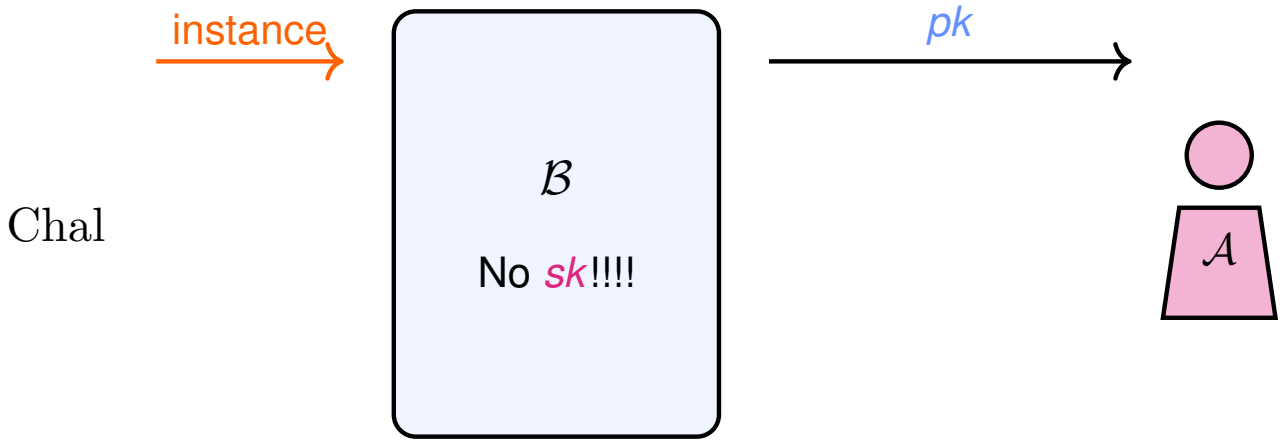
Three security notions for PKE

Same game (guess b from $ct^* \leftarrow \mathrm{Enc}(pk, m_b)$); they differ only in the access to $\mathcal{O}_{\mathrm{Dec}}(\cdot) = \mathrm{Dec}(sk, \cdot)$:

	$\mathcal{O}_{\mathrm{Dec}}$ *before* ct^*	$\mathcal{O}_{\mathrm{Dec}}$ *after* ct^*	Adversary
IND-CPA	×	×	passive
IND-CCA1	✓	×	lunchtime
IND-CCA2	✓	✓ $ct \neq ct^*$	active

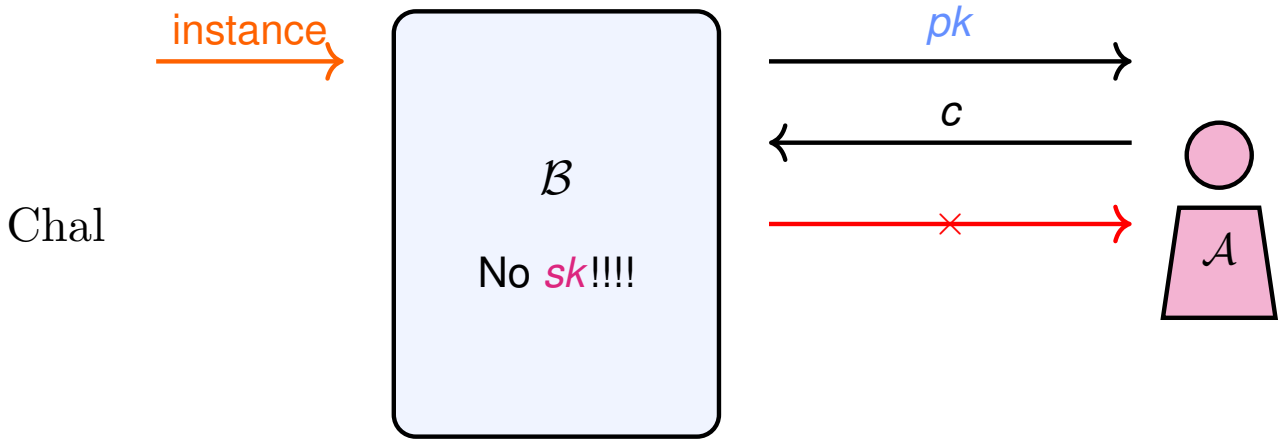
Goal: Construct IND-CCA2 secure PKE.

Challenge with IND-CCA



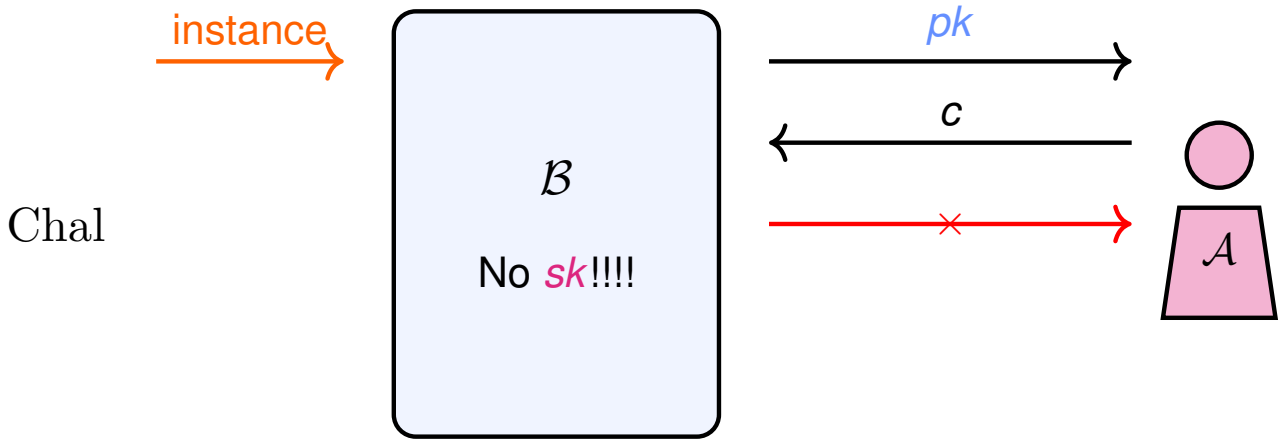
- So far: instance of the problem embedded in *pk*

Challenge with IND-CCA



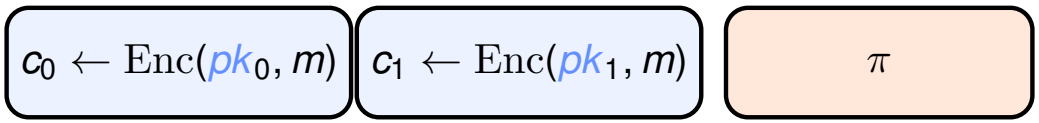
- So far: instance of the problem embedded in *pk*
- IND-CCA: how to answer decryption queries without *sk*?

Challenge with IND-CCA



- So far: instance of the problem embedded in *pk*
- IND-CCA: how to answer decryption queries without *sk*?
- Last lectures: *Naor-Yung*
  - Double encryption (one key always available to $\mathcal{B}$)
  - NIZK ensures consistency

Recall: Naor–Yung

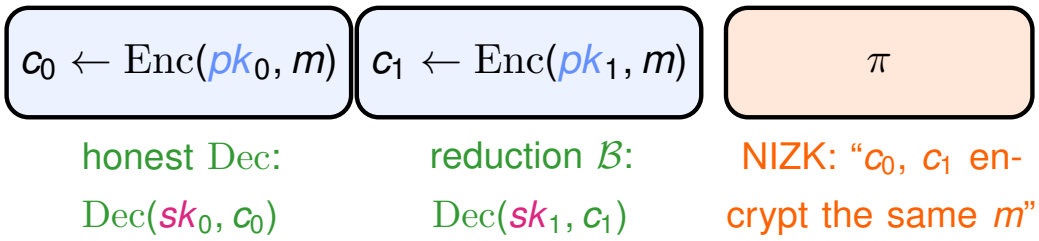


honest Dec: $\mathrm{Dec}(sk_0, c_0)$

reduction $\mathcal{B}$: $\mathrm{Dec}(sk_1, c_1)$

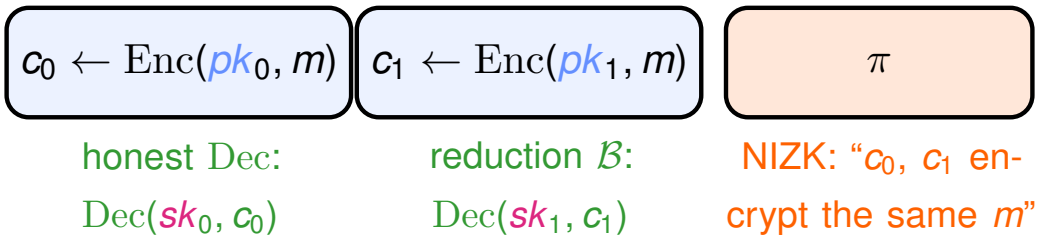
NIZK: "c_0, c_1 encrypt the same m"

Recall: Naor–Yung



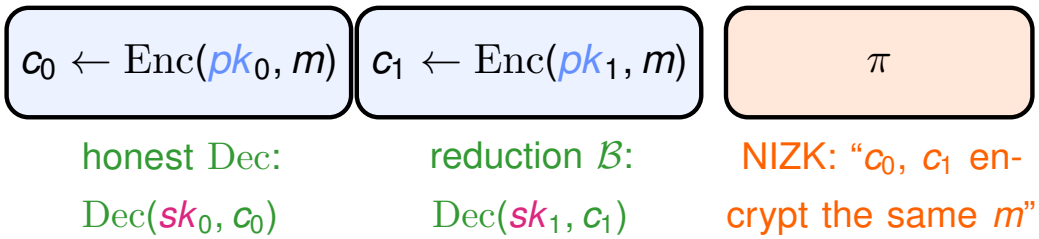
- Π IND-CPA + NIZK sound, ZK $\Rightarrow$ Π_{NY} IND-CCA1.
  (if NIZK *simulation-sound*, then Π_{NY} IND-CCA2).

Recall: Naor–Yung



- Π IND-CPA + NIZK sound, ZK $\Rightarrow$ Π_{NY} IND-CCA1.
  (if NIZK *simulation-sound*, then Π_{NY} IND-CCA2).
- Proof: hybrid argument; whoever answers $\mathcal{O}_{\mathrm{Dec}}$ knows *one key*, soundness makes decryption with both keys agree.

Recall: Naor–Yung



- Π IND-CPA + NIZK sound, ZK $\Rightarrow$ Π_{NY} IND-CCA1.
  (if NIZK *simulation-sound*, then Π_{NY} IND-CCA2).
- Proof: hybrid argument; whoever answers $\mathcal{O}_{\mathrm{Dec}}$ knows *one key*, soundness makes decryption with both keys agree.
- Generic and standard model
  - π is a NIZK for an NP statement about Enc.
  - **A feasibility result**, not a practical scheme.

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of NIZK.

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?
  - Decryptor needs to verify. Decryptor has sk!

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?
  - Decryptor needs to verify. Decryptor has *sk*!
  - *Designated verifier* consistency proof
  - Hope: such proofs *much cheaper* than a NIZK

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?
  - Decryptor needs to verify. Decryptor has *sk*!
  - *Designated verifier* consistency proof
  - Hope: such proofs *much cheaper* than a NIZK
- Main topic today:
  - How can the reduction $\mathcal{B}$ know the full *sk* as well?

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of the NIZK.

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of the NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of the NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?
  - Only the decryptor verifies – and the decryptor has sk!

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of the NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?
  - Only the decryptor verifies – and the decryptor has sk!
  - $\Rightarrow$ a *designated-verifier* proof of well-formedness suffices.

Today: getting rid of the NIZK

- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of the NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?
  - Only the decryptor verifies – and the decryptor has *sk*!
  - $\Rightarrow$ a *designated-verifier* proof of well-formedness suffices.
  - Such proofs will be *much cheaper* than a NIZK.

Today: getting rid of the NIZK

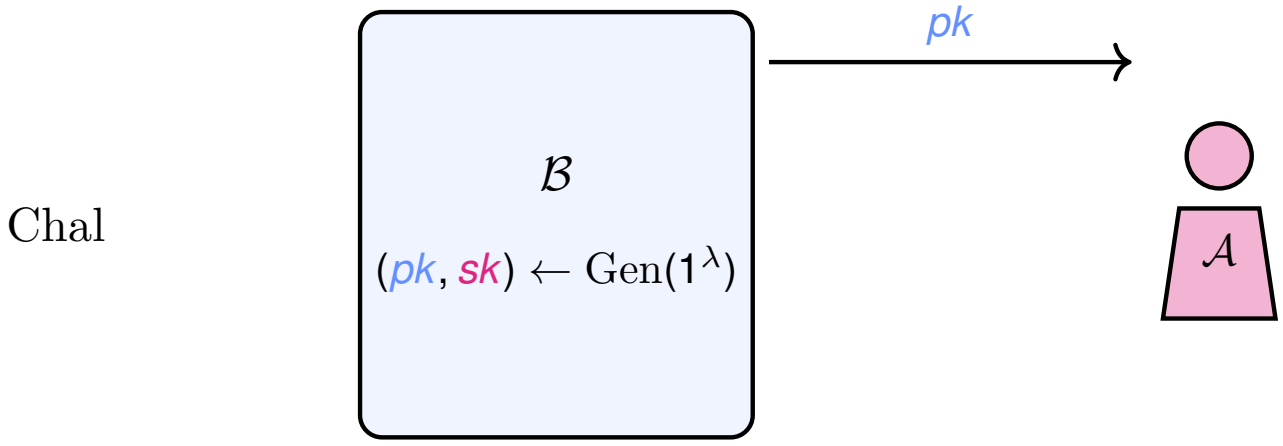
- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of the NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?
  - Only the decryptor verifies – and the decryptor has *sk*!
  - $\Rightarrow$ a *designated-verifier* proof of well-formedness suffices.
  - Such proofs will be *much cheaper* than a NIZK.
- The catch: checking needs *sk*, so the reduction $\mathcal{B}$ must hold *the sk* (not one of two, as in Naor–Yung).

Today: getting rid of the NIZK

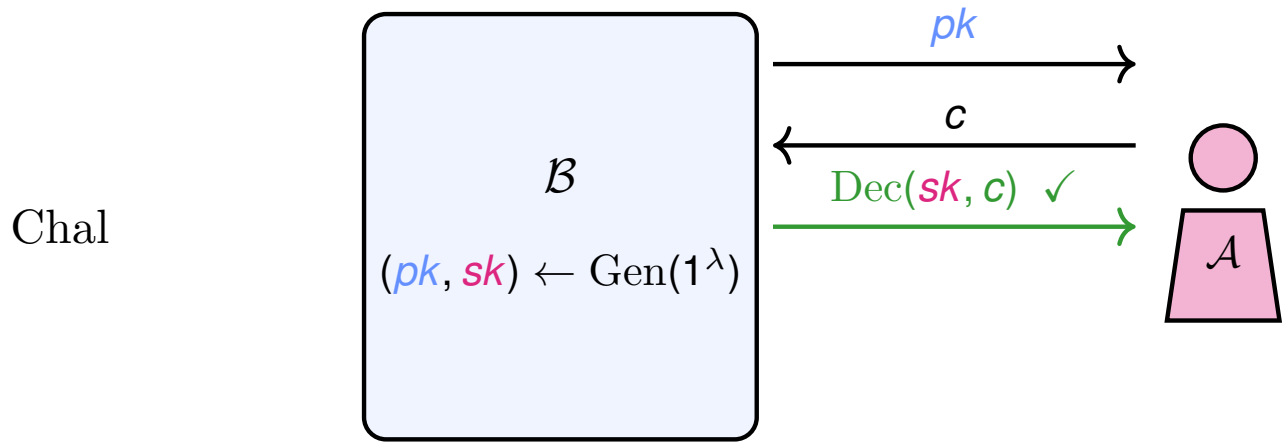
- We want a *practical* IND-CCA scheme $\Rightarrow$ get rid of the NIZK.
- **Observation (Cramer–Shoup):**
  - Do we really need public verifiability of the NIZK?
  - Only the decryptor verifies – and the decryptor has *sk*!
  - $\Rightarrow$ a *designated-verifier* proof of well-formedness suffices.
  - Such proofs will be *much cheaper* than a NIZK.
- The catch: checking needs *sk*, so the reduction $\mathcal{B}$ must hold *the sk* (not one of two, as in Naor–Yung).

Main question today: where does the hardness come from then?

Today's plan: let $\mathcal{B}$ know *the* key

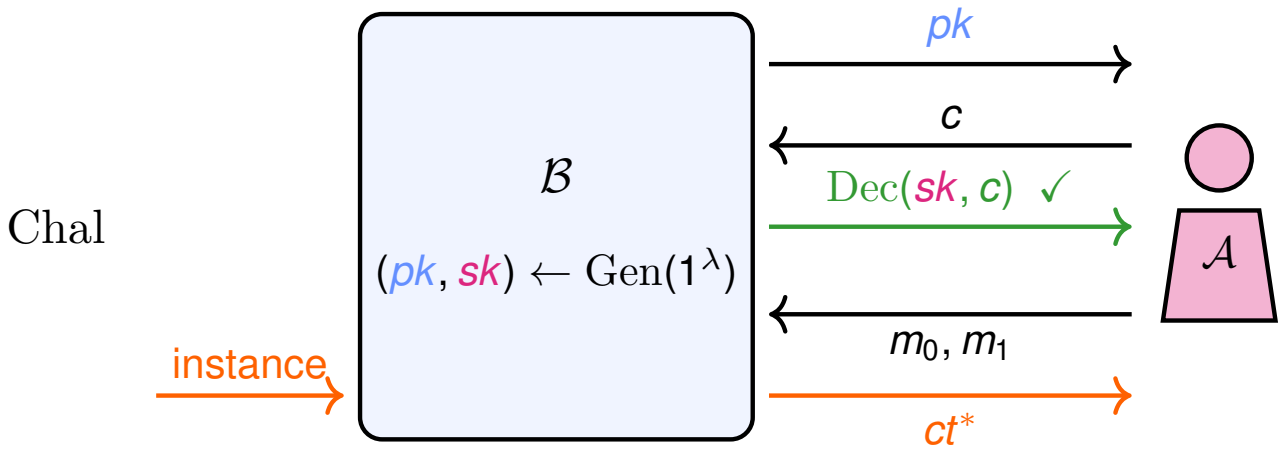


Today's plan: let $\mathcal{B}$ know *the* key



- If $\mathcal{B}$ picks *sk* itself
  - $\mathcal{B}$ *perfectly* simulates the real experiment.

Today's plan: let $\mathcal{B}$ know *the* key



- If $\mathcal{B}$ picks *sk* itself
  - $\mathcal{B}$ *perfectly* simulates the real experiment.
- But where does $\mathcal{B}$ embed their problem instance?
  - In the *challenge ciphertext*.
  - TODAY: HOW CAN WE DO THIS?

Recall plain ElGamal (hand written)

Recall plain ElGamal (hand written)

Question: Can one modify ElGamal, such that embedding of DDH instance possible in the challenge ciphertext?

Question: Can one modify ElGamal, such that embedding of DDH instance possible in the challenge ciphertext?

YES! We will do it more generically today.

Overview of our approach

1. Abstract DDH as a *Subset Membership Problem (SMP)*

   a set of “statements” with a hard-to-recognise subset of well-formed ones.

Overview of our approach

1. Abstract DDH as a *Subset Membership Problem (SMP)*

   a set of “statements” with a hard-to-recognise subset of well-formed ones.

2. Introduce *Smooth Projective Hash*
   - a keyed function on statements Λ_{sk}
   - on well-formed statements:
     - also computable with pk and a witness
   - on ill-formed statements:
     - unpredictable

Overview of our approach

1. Abstract DDH as a *Subset Membership Problem (SMP)*

   a set of “statements” with a hard-to-recognise subset of well-formed ones.

2. Introduce *Smooth Projective Hash*
   - a keyed function on statements Λ_{sk}
   - on well-formed statements:
     - also computable with pk and a witness
   - on ill-formed statements:
     - unpredictable

3. Encryption = one-time pad with the hash value $\Rightarrow$ IND-CPA.

Overview of our approach

1. Abstract DDH as a *Subset Membership Problem (SMP)*

   a set of “statements” with a hard-to-recognise subset of well-formed ones.

2. Introduce *Smooth Projective Hash*
   - a keyed function on statements Λ_{sk}
   - on well-formed statements:
     - also computable with pk and a witness
   - on ill-formed statements:
     - unpredictable

3. Encryption = one-time pad with the hash value $\Rightarrow$ IND-CPA.

4. Add a second hash as a *tag* $\Rightarrow$ IND-CCA1, with a full proof.

Overview of our approach

1. Abstract DDH as a *Subset Membership Problem (SMP)*

   a set of “statements” with a hard-to-recognise subset of well-formed ones.

2. Introduce *Smooth Projective Hash*
   - a keyed function on statements Λ_{sk}
   - on well-formed statements:
     - also computable with pk and a witness
   - on ill-formed statements:
     - unpredictable

3. Encryption = one-time pad with the hash value $\Rightarrow$ IND-CPA.

4. Add a second hash as a *tag* $\Rightarrow$ IND-CCA1, with a full proof.

Overview of our approach

1. Abstract DDH as a *Subset Membership Problem (SMP)*

   a set of “statements” with a hard-to-recognise subset of well-formed ones.

2. Introduce *Smooth Projective Hash*
   - a keyed function on statements Λ_{sk}
   - on well-formed statements:
     - also computable with pk and a witness
   - on ill-formed statements:
     - unpredictable

3. Encryption = one-time pad with the hash value $\Rightarrow$ IND-CPA.

4. Add a second hash as a *tag* $\Rightarrow$ IND-CCA1, with a full proof.

Every definition is followed by its DDH instantiation.

Outline

Recap: Lecture 4

Key idea

Technical tools

- Subset membership problems
- Smooth Projective Hashing

IND-CPA from a Smooth Projective Hash

Subset membership problems intuition (handwritten)

Subset membership problems

Definition 5.1: Subset Membership Problem (SMP)

Let $\mathcal{X}$ be a finite set, $\mathcal{R} \subset \mathcal{X} \times \mathcal{W}$ a relation, and $\mathcal{L} := \{ x \in \mathcal{X} : \exists w \, (x, w) \in \mathcal{R} \}$. We call $(\mathcal{X}, \mathcal{L}, \mathcal{R})$ a *subset membership problem* if

(All sets depend on the security parameter λ; kept implicit.)

Subset membership problems

Definition 5.1: Subset Membership Problem (SMP)

Let $\mathcal{X}$ be a finite set, $\mathcal{R} \subset \mathcal{X} \times \mathcal{W}$ a relation, and $\mathcal{L} := \{ x \in \mathcal{X} : \exists w \, (x, w) \in \mathcal{R} \}$. We call $(\mathcal{X}, \mathcal{L}, \mathcal{R})$ a *subset membership problem* if

- membership $x \in \mathcal{X}$ is efficiently decidable, and

(All sets depend on the security parameter λ; kept implicit.)

Subset membership problems

Definition 5.1: Subset Membership Problem (SMP)

Let $\mathcal{X}$ be a finite set, $\mathcal{R} \subset \mathcal{X} \times \mathcal{W}$ a relation, and $\mathcal{L} := \{ x \in \mathcal{X} : \exists w \, (x, w) \in \mathcal{R} \}$. We call $(\mathcal{X}, \mathcal{L}, \mathcal{R})$ a *subset membership problem* if

- membership $x \in \mathcal{X}$ is efficiently decidable, and
- *efficient sampling:* there are PPT algorithms that output $(x, w) \leftarrow_{\$} \mathcal{R}$ and $x \leftarrow_{\$} \mathcal{X} \setminus \mathcal{L}$ (uniformly).

(All sets depend on the security parameter λ; kept implicit.)

Subset membership problems

Definition 5.1: Subset Membership Problem (SMP)

Let $\mathcal{X}$ be a finite set, $\mathcal{R} \subset \mathcal{X} \times \mathcal{W}$ a relation, and $\mathcal{L} := \{ x \in \mathcal{X} : \exists w \, (x, w) \in \mathcal{R} \}$. We call $(\mathcal{X}, \mathcal{L}, \mathcal{R})$ a *subset membership problem* if

- membership $x \in \mathcal{X}$ is efficiently decidable, and
- *efficient sampling:* there are PPT algorithms that output $(x, w) \leftarrow_\$ \mathcal{R}$ and $x \leftarrow_\$ \mathcal{X} \setminus \mathcal{L}$ (uniformly).

(All sets depend on the security parameter λ; kept implicit.)

Terminology:

- $x \in \mathcal{L}$ are *well-formed* statements
- $x \in \mathcal{X} \setminus \mathcal{L}$ are *ill-formed* statements

Hardness of SMP

Intuition: SMP is hard if well-formed and ill-foremd statements are computationally indistinguishable.

Hardness of SMP

Intuition: SMP is hard if well-formed and ill-foremd statements are computationally indistinguishable.

Definition 5.2: hard subset membership problem

A subset membership problem is *hard* if for every PPT $\mathcal{A}$

$$\mathsf{Adv}^{\mathsf{SMP}}_{\mathcal{A}}(\lambda) \coloneqq \left| \Pr\left[\mathcal{A}(x) = 1 : (x, w) \leftarrow_{\$} \mathcal{R}\right] - \Pr\left[\mathcal{A}(x) = 1 : x \leftarrow_{\$} \mathcal{X} \setminus \mathcal{L}\right] \right|$$

is negligible in λ.

DDH as Hard SMP (handwritten)

DDH as Hard SMP - picture (handwritten)

DDH as HardSMP

DDH example

$\mathbb{G} = \langle g_1 \rangle$ of prime order p; a second generator $g_2 = g_1^{\omega}$, ω unknown to everyone. $\mathcal{X}_{\mathrm{DH}} \coloneqq \mathbb{G} \times \mathbb{G}$, $\mathcal{W} \coloneqq \mathbb{Z}_p$,
$\mathcal{R}_{\mathrm{DH}} \coloneqq \{((u_1, u_2), r) : u_1 = g_1^r \wedge u_2 = g_2^r\}$ $\mathcal{L}_{\mathrm{DH}} \coloneqq \{(g_1^r, g_2^r) : r \in \mathbb{Z}_p\}$

- $\mathcal{L}_{\mathrm{DH}}$: the *well-formed* pairs, p of them.
- $\mathcal{X} \setminus \mathcal{L}_{\mathrm{DH}}$: the *ill-formed* pairs $(g_1^{r_1}, g_2^{r_2})$, $r_1 \neq r_2$, $p^2 - p$ of them.

Sampling is easy either way.

DDH as HardSMP

DDH example

$\mathbb{G} = \langle g_1 \rangle$ of prime order p; a second generator $g_2 = g_1^{\omega}$, ω unknown to everyone. $\mathcal{X}_{\mathrm{DH}} \coloneqq \mathbb{G} \times \mathbb{G}$, $\mathcal{W} \coloneqq \mathbb{Z}_p$,
$\mathcal{R}_{\mathrm{DH}} \coloneqq \{((u_1, u_2), r) \colon u_1 = g_1^r \wedge u_2 = g_2^r\}$ $\mathcal{L}_{\mathrm{DH}} \coloneqq \{(g_1^r, g_2^r) : r \in \mathbb{Z}_p\}$

- $\mathcal{L}_{\mathrm{DH}}$: the *well-formed* pairs, p of them.
- $\mathcal{X} \setminus \mathcal{L}_{\mathrm{DH}}$: the *ill-formed* pairs $(g_1^{r_1}, g_2^{r_2})$, $r_1 \neq r_2$, $p^2 - p$ of them.

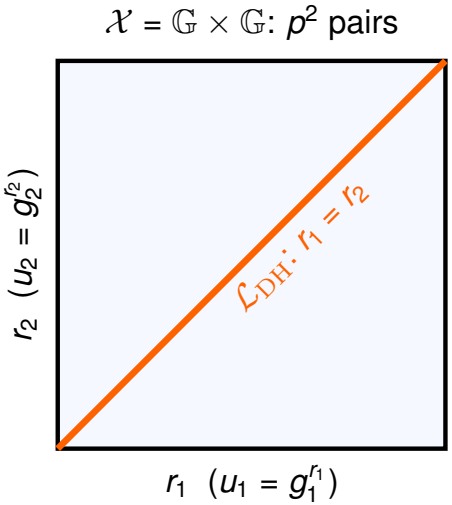
Sampling is easy either way.

$(\mathcal{X}_{\mathrm{DH}}, \mathcal{L}_{\mathrm{DH}}, \mathcal{R}_{\mathrm{DH}})$ **hard** $\Leftrightarrow$ **DDH.** Lecture 2:
$(g, g^x, g^y, g^{xy}) \approx_c (g, g^x, g^y, g^z)$. Rename $g_1 \coloneqq g$, $g_2 \coloneqq g^x$, $r \coloneqq y$:

$$(g_1, g_2, \underbrace{g_1^r, g_2^r}_{\in \mathcal{L}_{\mathrm{DH}}}) \quad \approx_c \quad (g_1, g_2, \underbrace{g_1^r, g_2^{r'}}_{\notin \mathcal{L}_{\mathrm{DH}} \text{ unless } r=r'}).$$

Well-formed and ill-formed pairs (picture)

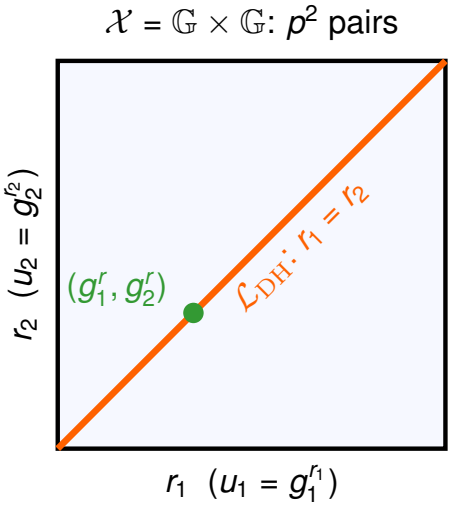
Every pair $(u_1, u_2) \in \mathbb{G} \times \mathbb{G}$ is $(g_1^{r_1}, g_2^{r_2})$ for a *unique* $(r_1, r_2) \in \mathbb{Z}_p^2$. In these *exponent coordinates*:



- The diagonal is $\mathcal{L}_{\mathrm{DH}}$.

Well-formed and ill-formed pairs (picture)

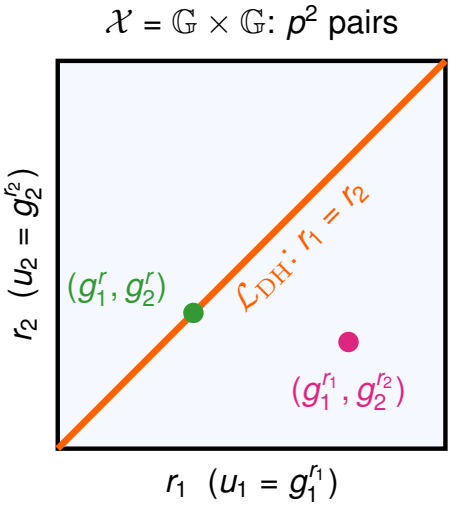
Every pair $(u_1, u_2) \in \mathbb{G} \times \mathbb{G}$ is $(g_1^{r_1}, g_2^{r_2})$ for a *unique* $(r_1, r_2) \in \mathbb{Z}_p^2$. In these *exponent coordinates*:



- The diagonal is $\mathcal{L}_{\mathrm{DH}}$.
- The honest encryptor samples *on* the line — it picks r, the witness.

Well-formed and ill-formed pairs (picture)

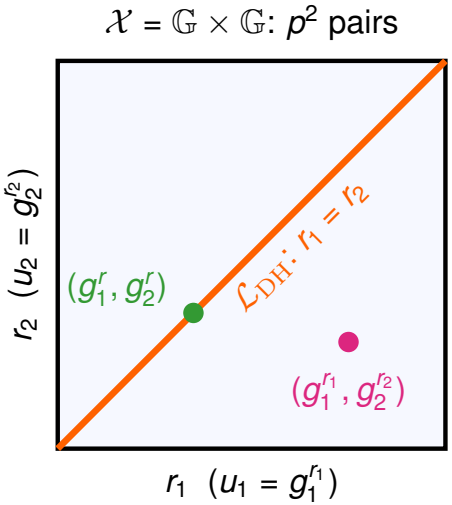
Every pair $(u_1, u_2) \in \mathbb{G} \times \mathbb{G}$ is $(g_1^{r_1}, g_2^{r_2})$ for a *unique* $(r_1, r_2) \in \mathbb{Z}_p^2$. In these *exponent coordinates*:



- The diagonal is $\mathcal{L}_{\mathrm{DH}}$.
- The honest encryptor samples *on* the line — it picks r, the witness.
- On the line $\iff$ $u_2 = u_1^{\omega}$. Deciding this is DDH; easy with ω, but nobody has ω.

Well-formed and ill-formed pairs (picture)

Every pair $(u_1, u_2) \in \mathbb{G} \times \mathbb{G}$ is $(g_1^{r_1}, g_2^{r_2})$ for a *unique* $(r_1, r_2) \in \mathbb{Z}_p^2$. In these *exponent coordinates*:



- The diagonal is $\mathcal{L}_{\mathrm{DH}}$.
- The honest encryptor samples *on* the line — it picks r, the witness.
- On the line $\iff$ $u_2 = u_1^{\omega}$. Deciding this is DDH; easy with ω, but nobody has ω.

⚠ The coordinates (r_1, r_2) are *not computable* (discrete log). The picture shows how things *are*, not what anyone *sees*.

Smooth Projective Hashing - intuition (handwritten)

Projective hashing

Definition 5.3: projective hash for $(\mathcal{X}, \mathcal{L}, \mathcal{R})$

Let $(\mathcal{X}, \mathcal{L}, \mathcal{R})$ be a subset membership problem. A *projective hash* for it consists of

- a key space $\mathcal{K}$, a projected-key space $\mathcal{P}$, and a value space $\mathcal{V}$,

We require that μ and Λ_{sk} are efficiently computable.

Projective hashing

Definition 5.3: projective hash for $(\mathcal{X}, \mathcal{L}, \mathcal{R})$

Let $(\mathcal{X}, \mathcal{L}, \mathcal{R})$ be a subset membership problem. A *projective hash* for it consists of

- a key space $\mathcal{K}$, a projected-key space $\mathcal{P}$, and a value space $\mathcal{V}$,
- a family of functions $\Lambda_{sk} : \mathcal{X} \to \mathcal{V}$, one for each $sk \in \mathcal{K}$,

We require that μ and Λ_{sk} are efficiently computable.

Projective hashing

Definition 5.3: projective hash for $(\mathcal{X}, \mathcal{L}, \mathcal{R})$

Let $(\mathcal{X}, \mathcal{L}, \mathcal{R})$ be a subset membership problem. A *projective hash* for it consists of

- a key space $\mathcal{K}$, a projected-key space $\mathcal{P}$, and a value space $\mathcal{V}$,
- a family of functions $\Lambda_{sk} : \mathcal{X} \to \mathcal{V}$, one for each $sk \in \mathcal{K}$,
- a *projection* $\mu : \mathcal{K} \to \mathcal{P}$, $pk := \mu(sk)$, and

We require that μ and Λ_{sk} are efficiently computable.

Projective hashing

Definition 5.3: projective hash for $(\mathcal{X}, \mathcal{L}, \mathcal{R})$

Let $(\mathcal{X}, \mathcal{L}, \mathcal{R})$ be a subset membership problem. A *projective hash* for it consists of

- a key space $\mathcal{K}$, a projected-key space $\mathcal{P}$, and a value space $\mathcal{V}$,
- a family of functions $\Lambda_{sk} : \mathcal{X} \to \mathcal{V}$, one for each $sk \in \mathcal{K}$,
- a *projection* $\mu : \mathcal{K} \to \mathcal{P}$, $pk := \mu(sk)$, and
- a PPT algorithm $\mathrm{Pub} : \mathcal{P} \times \mathcal{R} \to \mathcal{V}$ such that for all $sk \in \mathcal{K}$ and all $(x, w) \in \mathcal{R}$:

$$\Lambda_{sk}(x) = \mathrm{Pub}(\mu(sk), x, w)$$

We require that μ and Λ_{sk} are efficiently computable.

Projective hashing

Projective hash gives *two ways* to the same value:

- *privately*, with *sk* for *every* $x \in \mathcal{X}$;
- *publicly*, with *pk* and a witness only for $x \in \mathcal{L}$.

Projective hashing

Projective hash gives *two ways* to the same value:

- *privately*, with *sk* for *every* $x \in \mathcal{X}$;
- *publicly*, with *pk* and a witness only for $x \in \mathcal{L}$.

Projective means: **in** $\mathcal{L}$, *pk* determines the hash value.

Smooth Projective Hash

What happens outside of $\mathcal{L}$?

Smooth Projective Hash

What happens outside of $\mathcal{L}$?

Definition 5.4: Smooth Projective Hash

Let $(\Lambda, \mu, \mathrm{Pub})$ be a projective hash for $(\mathcal{X}, \mathcal{L}, \mathcal{R})$. For $x \in \mathcal{X}$ consider the two distributions on $\mathcal{P} \times \mathcal{V}$

$$D_x \coloneqq \{(\mu(sk),\ \Lambda_{sk}(x)) : sk \leftarrow_\$ \mathcal{K}\}$$
$$U \coloneqq \{(\mu(sk),\ v) : sk \leftarrow_\$ \mathcal{K},\ v \leftarrow_\$ \mathcal{V}\}.$$

The hash is *smooth* if $D_x = U$ for every $x \in \mathcal{X} \setminus \mathcal{L}$.

Smooth Projective Hash

What happens outside of $\mathcal{L}$?

Definition 5.4: Smooth Projective Hash

Let $(\Lambda, \mu, \mathrm{Pub})$ be a projective hash for $(\mathcal{X}, \mathcal{L}, \mathcal{R})$. For $x \in \mathcal{X}$ consider the two distributions on $\mathcal{P} \times \mathcal{V}$

$$D_x \coloneqq \{(\mu(sk),\ \Lambda_{sk}(x)) : sk \leftarrow_\$ \mathcal{K}\}$$
$$U \coloneqq \{(\mu(sk),\ v) : sk \leftarrow_\$ \mathcal{K},\ v \leftarrow_\$ \mathcal{V}\}.$$

The hash is *smooth* if $D_x = U$ for every $x \in \mathcal{X} \setminus \mathcal{L}$.

In words: **outside of** $\mathcal{L}$, the hash value is *uniform* even given *pk*.

The DDH instance: a projective hash (handwritten)

The DDH instance: smoothness (handwritten)

The DDH instance: a projective hash

DDH example

$\mathcal{K} := \mathbb{Z}_p^2$, $\mathcal{P} := \mathbb{G}$, $\mathcal{V} := \mathbb{G}$. For $sk = (k_1, k_2)$:

$\mu(sk) := g_1^{k_1} g_2^{k_2}$, $\Lambda_{sk}(u_1, u_2) := u_1^{k_1} u_2^{k_2}$, $\mathrm{Pub}(pk, (u_1, u_2), r) := pk^{\,r}$.

The DDH instance: a projective hash

DDH example

$\mathcal{K} := \mathbb{Z}_p^2$, $\mathcal{P} := \mathbb{G}$, $\mathcal{V} := \mathbb{G}$. For $sk = (k_1, k_2)$:

$\mu(sk) := g_1^{k_1} g_2^{k_2}$, $\Lambda_{sk}(u_1, u_2) := u_1^{k_1} u_2^{k_2}$, $\mathrm{Pub}(pk, (u_1, u_2), r) := pk^r$.

Projective (Def. 5.3): for $((u_1, u_2), r) \in \mathcal{R}$, i.e. $(u_1, u_2) = (g_1^r, g_2^r)$,

$$\Lambda_{sk}(u_1, u_2) = g_1^{rk_1} g_2^{rk_2} = (g_1^{k_1} g_2^{k_2})^r = pk^r = \mathrm{Pub}(pk, (u_1, u_2), r). \quad \checkmark$$

The DDH instance: smoothness

DDH example

$\mathcal{K} := \mathbb{Z}_p^2$, $\mathcal{P} := \mathbb{G}$, $\mathcal{V} := \mathbb{G}$. For $sk = (k_1, k_2)$:

$\mu(sk) := g_1^{k_1} g_2^{k_2}$, $\Lambda_{sk}(u_1, u_2) := u_1^{k_1} u_2^{k_2}$, $\mathrm{Pub}(pk, (u_1, u_2), r) := pk^r$.

Smooth (Def. 5.4)? First: what does pk reveal about sk?

The DDH instance: smoothness

DDH example

$\mathcal{K} := \mathbb{Z}_p^2$, $\mathcal{P} := \mathbb{G}$, $\mathcal{V} := \mathbb{G}$. For $sk = (k_1, k_2)$:

$\mu(sk) := g_1^{k_1} g_2^{k_2}$, $\Lambda_{sk}(u_1, u_2) := u_1^{k_1} u_2^{k_2}$, $\mathrm{Pub}(pk, (u_1, u_2), r) := pk^{\,r}$.

Smooth (Def. 5.4)? First: what does pk reveal about sk?

$$pk = \mu(sk) = g_1^{k_1} g_2^{k_2} = g_1^{k_1 + \omega k_2}.$$

- Information theoretically, pk reveals $s := k_1 + \omega k_2 \mod p$
- That is *one linear equation* in two unknowns
- Hence, there are p secret keys consistent with pk

The DDH instance is smooth

Lemma 5.1: Smoothness of DDH projective hash

The DDH projective hash is smooth.

The DDH instance is smooth

Lemma 5.1: Smoothness of DDH projective hash

The DDH projective hash is smooth.

Proof. Fix $(u_1, u_2) = (g_1^{r_1}, g_2^{r_2}) \notin \mathcal{L}_{\mathrm{DH}}$, i.e. $r_1 \neq r_2$; $sk = (k_1, k_2) \leftarrow_{\$} \mathbb{Z}_p^2$.

The DDH instance is smooth

Lemma 5.1: Smoothness of DDH projective hash

The DDH projective hash is smooth.

Proof. Fix $(u_1, u_2) = (g_1^{r_1}, g_2^{r_2}) \notin \mathcal{L}_{\mathrm{DH}}$, i.e. $r_1 \neq r_2$; $sk = (k_1, k_2) \leftarrow_{\$} \mathbb{Z}_p^2$.

- $pk = g_1^{k_1 + \omega k_2}$ reveals exactly $s := k_1 + \omega k_2$
- Given pk, k_2 is still uniform and $k_1 = s - \omega k_2$.

The DDH instance is smooth

Lemma 5.1: Smoothness of DDH projective hash

The DDH projective hash is smooth.

Proof. Fix $(u_1, u_2) = (g_1^{r_1}, g_2^{r_2}) \notin \mathcal{L}_{\mathrm{DH}}$, i.e. $r_1 \neq r_2$; $sk = (k_1, k_2) \leftarrow_{\$} \mathbb{Z}_p^2$.

- $pk = g_1^{k_1 + \omega k_2}$ reveals exactly $s := k_1 + \omega k_2$
- Given pk, k_2 is still uniform and $k_1 = s - \omega k_2$.
- Substituting:

$$\log \Lambda_{sk}(u_1, u_2) = r_1 k_1 + \omega r_2 k_2 = \underbrace{r_1 s}_{\text{fixed by } pk} + \underbrace{\omega(r_2 - r_1)}_{\neq 0} \cdot k_2,$$

  a non-constant affine function of the uniform k_2, hence uniform in $\mathbb{Z}_p$.

The DDH instance is smooth

Lemma 5.1: Smoothness of DDH projective hash

The DDH projective hash is smooth.

Proof. Fix $(u_1, u_2) = (g_1^{r_1}, g_2^{r_2}) \notin \mathcal{L}_{\mathrm{DH}}$, i.e. $r_1 \neq r_2$; $sk = (k_1, k_2) \leftarrow_{\$} \mathbb{Z}_p^2$.

- $pk = g_1^{k_1 + \omega k_2}$ reveals exactly $s := k_1 + \omega k_2$
- Given pk, k_2 is still uniform and $k_1 = s - \omega k_2$.
- Substituting:

$$\log \Lambda_{sk}(u_1, u_2) = r_1 k_1 + \omega r_2 k_2 = \underbrace{r_1 s}_{\text{fixed by } pk} + \underbrace{\omega(r_2 - r_1)}_{\neq 0} \cdot k_2,$$

  a non-constant affine function of the uniform k_2, hence uniform in $\mathbb{Z}_p$.
- So $\Lambda_{sk}(u_1, u_2)$ is uniform in $\mathbb{G}$ given pk: $D_x = U$. □

The DDH instance is smooth

Lemma 5.1: Smoothness of DDH projective hash

The DDH projective hash is smooth.

Proof. Fix $(u_1, u_2) = (g_1^{r_1}, g_2^{r_2}) \notin \mathcal{L}_{\mathrm{DH}}$, i.e. $r_1 \neq r_2$; $sk = (k_1, k_2) \leftarrow_{\$} \mathbb{Z}_p^2$.

- $pk = g_1^{k_1 + \omega k_2}$ reveals exactly $s := k_1 + \omega k_2$
- Given pk, k_2 is still uniform and $k_1 = s - \omega k_2$.
- Substituting:

$$\log \Lambda_{sk}(u_1, u_2) = r_1 k_1 + \omega r_2 k_2 = \underbrace{r_1 s}_{\text{fixed by } pk} + \underbrace{\omega(r_2 - r_1)}_{\neq 0} \cdot k_2,$$

  a non-constant affine function of the uniform k_2, hence uniform in $\mathbb{Z}_p$.
- So $\Lambda_{sk}(u_1, u_2)$ is uniform in $\mathbb{G}$ given pk: $D_x = U$. □

For $r_1 = r_2$ the coefficient of k_2 vanishes: all keys agree (projectivity). **No assumption used.**

Summary: the tool

	generic	DDH instance
statements	$x \in \mathcal{X}$	$(u_1, u_2) \in \mathbb{G} \times \mathbb{G}$
well-formed	$x \in \mathcal{L}$, witness w	(g_1^r, g_2^r), witness r
hard	$\mathcal{L} \approx \mathcal{X} \setminus \mathcal{L}$	DDH
secret key	$sk \in \mathcal{K}$	$(k_1, k_2) \in \mathbb{Z}_p^2$
public key	$pk = \mu(sk)$	$g_1^{k_1} g_2^{k_2}$
private evaluation	$\Lambda_{sk}(x) \in \mathcal{V}$	$u_1^{k_1} u_2^{k_2} \in \mathbb{G}$
public evaluation	$\mathrm{Pub}(pk, x, w)$, $x \in \mathcal{L}$	pk^r
projective	in $\mathcal{L}$: value determined by pk	$= g_1^{rs}$ for all consistent keys
smooth	in $\mathcal{X} \setminus \mathcal{L}$: value uniform given pk	$\log = r_1 s + \omega(r_2 - r_1)k_2$

Outline

Recap: Lecture 4

Key idea

Technical tools

- Subset membership problems
- Smooth Projective Hashing

IND-CPA from a Smooth Projective Hash

The hash value as a one-time pad

Building blocks:

- A subset membership problem SMP = $(\mathcal{X}, \mathcal{L}, \mathcal{R})$,
- A smooth projective hash $(\Lambda, \mu, \mathrm{Pub})$ for SMP with key space $\mathcal{K}$, projection space $\mathcal{P}$ and values in a *group* $\mathcal{V}$.

$\Pi_{\mathrm{pad}} = (\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ `with message space` $\mathcal{M} = \mathcal{V}$

$\mathrm{Gen}(1^\lambda)$: $sk \leftarrow_\$ \mathcal{K}$, $pk := \mu(sk)$

The hash value as a one-time pad

Building blocks:

- A subset membership problem SMP = $(\mathcal{X}, \mathcal{L}, \mathcal{R})$,
- A smooth projective hash $(\Lambda, \mu, \mathrm{Pub})$ for SMP with key space $\mathcal{K}$, projection space $\mathcal{P}$ and values in a *group* $\mathcal{V}$.

$\Pi_{\mathrm{pad}} = (\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ `with message space` $\mathcal{M} = \mathcal{V}$

$\mathrm{Gen}(1^\lambda)$: $sk \leftarrow_\$ \mathcal{K}$, $pk := \mu(sk)$

$\mathrm{Enc}(pk, m)$: sample $(x, w) \leftarrow_\$ \mathcal{R}$; $e := m \cdot \mathrm{Pub}(pk, x, w)$

output $ct := (x, e)$

The hash value as a one-time pad

Building blocks:

- A subset membership problem SMP = $(\mathcal{X}, \mathcal{L}, \mathcal{R})$,
- A smooth projective hash $(\Lambda, \mu, \mathrm{Pub})$ for SMP with key space $\mathcal{K}$, projection space $\mathcal{P}$ and values in a *group* $\mathcal{V}$.

$\Pi_{\mathrm{pad}} = (\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ `with message space` $\mathcal{M} = \mathcal{V}$

$\mathrm{Gen}(1^\lambda)$: $sk \leftarrow_\$ \mathcal{K}$, $pk := \mu(sk)$

$\mathrm{Enc}(pk, m)$: sample $(x, w) \leftarrow_\$ \mathcal{R}$; $e := m \cdot \mathrm{Pub}(pk, x, w)$

output $ct := (x, e)$

$\mathrm{Dec}(sk, ct)$: parse $ct = (x, e)$; output $e \,/\, \Lambda_{sk}(x)$

The hash value as a one-time pad

Building blocks:

- A subset membership problem SMP = $(\mathcal{X}, \mathcal{L}, \mathcal{R})$,
- A smooth projective hash $(\Lambda, \mu, \mathrm{Pub})$ for SMP with key space $\mathcal{K}$, projection space $\mathcal{P}$ and values in a *group* $\mathcal{V}$.

$\Pi_{\mathrm{pad}} = (\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ `with message space` $\mathcal{M} = \mathcal{V}$

$\mathrm{Gen}(1^\lambda)$: $sk \leftarrow_\$ \mathcal{K}$, $pk := \mu(sk)$

$\mathrm{Enc}(pk, m)$: sample $(x, w) \leftarrow_\$ \mathcal{R}$; $e := m \cdot \mathrm{Pub}(pk, x, w)$
output $ct := (x, e)$

$\mathrm{Dec}(sk, ct)$: parse $ct = (x, e)$; output $e \,/\, \Lambda_{sk}(x)$

Enc evaluates the hash *publicly* (it has w), Dec *privately* (it has sk).

The DDH instance: ElGamal with two generators

DDH example

Gen: $k_1, k_2 \leftarrow_\$ \mathbb{Z}_p$, $h := g_1^{k_1} g_2^{k_2}$; $pk = h$, $sk = (k_1, k_2)$

$\mathrm{Enc}(pk, m)$: $r \leftarrow_\$ \mathbb{Z}_p$; $ct := (g_1^r, g_2^r, m \cdot h^r)$

$\mathrm{Dec}(sk, ct)$: parse $ct = (u_1, u_2, e)$; output $e \,/\, (u_1^{k_1} u_2^{k_2})$

The DDH instance: ElGamal with two generators

DDH example

Gen: $k_1, k_2 \leftarrow_\$ \mathbb{Z}_p$, $h := g_1^{k_1} g_2^{k_2}$; $pk = h$, $sk = (k_1, k_2)$

$\mathrm{Enc}(pk, m)$: $r \leftarrow_\$ \mathbb{Z}_p$; $ct := (g_1^r, g_2^r, m \cdot h^r)$

$\mathrm{Dec}(sk, ct)$: parse $ct = (u_1, u_2, e)$; output $e / (u_1^{k_1} u_2^{k_2})$

- Compared to ElGamal: (g_1^r, g_2^r) in place of g^r, and a two-dimensional secret key.

The DDH instance: ElGamal with two generators

DDH example

Gen: $k_1, k_2 \leftarrow_\$ \mathbb{Z}_p$, $h := g_1^{k_1} g_2^{k_2}$; $pk = h$, $sk = (k_1, k_2)$

$\mathrm{Enc}(pk, m)$: $r \leftarrow_\$ \mathbb{Z}_p$; $ct := (g_1^r, g_2^r, m \cdot h^r)$

$\mathrm{Dec}(sk, ct)$: parse $ct = (u_1, u_2, e)$; output $e \,/\, (u_1^{k_1} u_2^{k_2})$

- Compared to ElGamal: (g_1^r, g_2^r) in place of g^r, and a two-dimensional secret key.
- On honest ciphertexts it behaves exactly like ElGamal. Looks like a pointless variant.

The DDH instance: ElGamal with two generators

DDH example

Gen: $k_1, k_2 \leftarrow_\$ \mathbb{Z}_p$, $h := g_1^{k_1} g_2^{k_2}$; $pk = h$, $sk = (k_1, k_2)$

$\mathrm{Enc}(pk, m)$: $r \leftarrow_\$ \mathbb{Z}_p$; $ct := (g_1^r, g_2^r, m \cdot h^r)$

$\mathrm{Dec}(sk, ct)$: parse $ct = (u_1, u_2, e)$; output $e \,/\, (u_1^{k_1} u_2^{k_2})$

- Compared to ElGamal: (g_1^r, g_2^r) in place of g^r, and a two-dimensional secret key.
- On honest ciphertexts it behaves exactly like ElGamal. Looks like a pointless variant.
- The difference is in what the *secret key* can do: evaluate the hash on *ill-formed* pairs, where pk says nothing.

IND-CPA security

Theorem 5.2: IND-CPA security of Π_{pad}

If the subset membership problem is hard and the hash is smooth and projective, then Π_{pad} is IND-CPA secure: $\mathsf{Adv}^{\text{IND-CPA}}_{\Pi_{\mathrm{pad}},\mathcal{A}} \leq 2 \cdot \mathsf{Adv}^{\text{SMP}}_{\mathcal{B}}$.

Proof idea of Theorem 5.2 (handwritten)

Proof of Theorem 5.2: the games

game	x^* from	e^*	$G_{i-1} \to G_i$ via	ϵ_i
G_0	$\mathcal{L}$	$m_0 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 0$	
G_7	$\mathcal{L}$	$m_1 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 1$	

Proof of Theorem 5.2: the games

game	x^* from	e^*	$G_{i-1} \to G_i$ via	ϵ_i
G_0	$\mathcal{L}$	$m_0 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 0$	
G_1	$\mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	projectivity	0
G_7	$\mathcal{L}$	$m_1 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 1$	

Proof of Theorem 5.2: the games

game	x^* from	e^*	$G_{i-1} \to G_i$ via	ϵ_i
G_0	$\mathcal{L}$	$m_0 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 0$	
G_1	$\mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	projectivity	0
G_2	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_1}$
G_7	$\mathcal{L}$	$m_1 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 1$	

Proof of Theorem 5.2: the games

game	x^* from	e^*	$G_{i-1} \to G_i$ via	ϵ_i
G_0	$\mathcal{L}$	$m_0 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 0$	
G_1	$\mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	projectivity	0
G_2	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_1}$
G_3	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot v$, $v \leftarrow_\$ \mathcal{V}$	smoothness	0
G_7	$\mathcal{L}$	$m_1 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 1$	

Proof of Theorem 5.2: the games

game	x^* from	e^*	$G_{i-1} \to G_i$ via	ϵ_i
G_0	$\mathcal{L}$	$m_0 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 0$	
G_1	$\mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	projectivity	0
G_2	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_1}$
G_3	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot v,\ v \leftarrow_\$ \mathcal{V}$	smoothness	0
G_4	$\mathcal{X} \setminus \mathcal{L}$	$m_1 \cdot v,\ v \leftarrow_\$ \mathcal{V}$	one-time pad in $\mathcal{V}$	0
G_7	$\mathcal{L}$	$m_1 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 1$	

Proof of Theorem 5.2: the games

game	x^* from	e^*	$G_{i-1} \to G_i$ via	ϵ_i
G_0	$\mathcal{L}$	$m_0 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 0$	
G_1	$\mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	projectivity	0
G_2	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_1}$
G_3	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot v,\ v \leftarrow_\$ \mathcal{V}$	smoothness	0
G_4	$\mathcal{X} \setminus \mathcal{L}$	$m_1 \cdot v,\ v \leftarrow_\$ \mathcal{V}$	one-time pad in $\mathcal{V}$	0
G_5	$\mathcal{X} \setminus \mathcal{L}$	$m_1 \cdot \Lambda_{sk}(x^*)$	smoothness	0
G_7	$\mathcal{L}$	$m_1 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 1$	

Proof of Theorem 5.2: the games

game	x^* from	e^*	$G_{i-1} \to G_i$ via	ϵ_i
G_0	$\mathcal{L}$	$m_0 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 0$	
G_1	$\mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	projectivity	0
G_2	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_1}$
G_3	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot v,\ v \leftarrow_\$ \mathcal{V}$	smoothness	0
G_4	$\mathcal{X} \setminus \mathcal{L}$	$m_1 \cdot v,\ v \leftarrow_\$ \mathcal{V}$	one-time pad in $\mathcal{V}$	0
G_5	$\mathcal{X} \setminus \mathcal{L}$	$m_1 \cdot \Lambda_{sk}(x^*)$	smoothness	0
G_6	$\mathcal{L}$	$m_1 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_2}$
G_7	$\mathcal{L}$	$m_1 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 1$	

Proof of Theorem 5.2: the games

game	x^* from	e^*	$G_{i-1} \to G_i$ via	ϵ_i
G_0	$\mathcal{L}$	$m_0 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 0$	
G_1	$\mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	projectivity	0
G_2	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_1}$
G_3	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot v,\ v \leftarrow_{\$} \mathcal{V}$	smoothness	0
G_4	$\mathcal{X} \setminus \mathcal{L}$	$m_1 \cdot v,\ v \leftarrow_{\$} \mathcal{V}$	one-time pad in $\mathcal{V}$	0
G_5	$\mathcal{X} \setminus \mathcal{L}$	$m_1 \cdot \Lambda_{sk}(x^*)$	smoothness	0
G_6	$\mathcal{L}$	$m_1 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_2}$
G_7	$\mathcal{L}$	$m_1 \cdot \mathrm{Pub}(pk, x^*, w^*)$	projectivity	0

Proof of Theorem 5.2: the games

game	x^* from	e^*	$G_{i-1} \to G_i$ via	ϵ_i
G_0	$\mathcal{L}$	$m_0 \cdot \mathrm{Pub}(pk, x^*, w^*)$	IND-CPA, $b = 0$	
G_1	$\mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	projectivity	0
G_2	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_1}$
G_3	$\mathcal{X} \setminus \mathcal{L}$	$m_0 \cdot v,\ v \leftarrow_\$ \mathcal{V}$	smoothness	0
G_4	$\mathcal{X} \setminus \mathcal{L}$	$m_1 \cdot v,\ v \leftarrow_\$ \mathcal{V}$	one-time pad in $\mathcal{V}$	0
G_5	$\mathcal{X} \setminus \mathcal{L}$	$m_1 \cdot \Lambda_{sk}(x^*)$	smoothness	0
G_6	$\mathcal{L}$	$m_1 \cdot \Lambda_{sk}(x^*)$	SMP	$\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_2}$
G_7	$\mathcal{L}$	$m_1 \cdot \mathrm{Pub}(pk, x^*, w^*)$	projectivity	0

Only two hops cost something:

$$\mathrm{Adv}^{\text{IND-CPA}}_{\Pi_{\text{pad}},\mathcal{A}} = |\Pr[G_7 \to 1] - \Pr[G_0 \to 1]| \leq \mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_1} + \mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}_2} \leq 2\,\mathrm{Adv}^{\mathrm{SMP}}_{\mathcal{B}}.$$

Hop $G_1 \rightarrow G_2$ (handwritten)